Using Neuroevolution for Intrusion Detection
System Test Plan
Version <2.0>
04/15/2025

Document Control

Distribution List

The following list of people will receive a copy of this document every time a new version of this document becomes available:

Teaching assistants:

Jose Alejandro Gonzalez Nunez

Customer(s): Dr. M. Ilhan Akbas

Project team members:

Regan Bossie, Tyler Donay, Megan Krajewski, Richard Pepe

Change Summary

The following table details changes made between versions of this document:

Version	Date	Modifier	Description
1.0	12/05/2024		Initial Release
1.1	04/10/2025	Donay	Formatting changes
1.2	04/14/2025	Donay	Rewrote 2.5 Risks & Contingencies, added the 5 major risks and associated contingencies
1.3	04/15/2025	Donay	Rewrote 2.1 to add descriptions to each subsection and removed associated requirements, fixed Test Schedule and Test Plan structure.
1.4	04/15/2025	Donay, Bossie, Pepe	Filled out test schedule, plan, and test cases.
2.0	04/15/25		Final Submission

Table of Contents

Document Control ii
Distribution List ii
Change Summary ii
1. Introduction 1
1.1. Purpose 1
1.2. Scope 1
1.3. System Overview 1
1.4. Testing Approach Overview 2
1.5. Testing Entrance Criteria 3
1.6. Document Overview 3
1.7. References 3
2. Testing Approach 5
2.1. Testing Types 5
2.1.1. Functional Testing 5
2.1.2. Compatibility Testing 5
2.1.3. Dataset Validation Testing 6
2.1.4. Usability Testing 6
2.1.5. Algorithm Testing 6
2.2. Testing Suspension Criteria and Resumption Requirements 7
2.2.1. Suspension Criteria 7
2.2.2. Resumption Requirements 7
2.3. Testing Environment 7
2.4. Testing Assumptions 7
2.5. Testing Risks and Contingencies 8
2.6. Test Plan 9
3. Test Schedule 10
4. Traceability Matrix and Defect Tracking 10
4.1. Traceability Matrix 10
4.2. Defect Severity Definitions 13
5. Test Cases 14
1. Functional Testing 14
Test Case 1.1: Verify Dataset Management 15
Test Case 1.2: Test Analysis and Reporting Functionality 17

Test Case 1.3: Validate Evolutionary Behavior 18
5.1.2. 2. Compatibility Testing 21
5.1.3. 3. Dataset Validation Testing 24
5.1.4. 4. Usability Testing 29
5.1.5. 5. Algorithm Testing 31

1. Introduction

1.1. Purpose

This document is the System Test Plan for the project: Using Neuroevolution for Intrusion Detection. It will detail the test approach, test schedule, traceability, and test cases for the project.

This project's goal is to determine the value of a neuroevolution machine learning approach to solving cybersecurity related issues. To do this neuroevolution techniques will be applied to develop an anomalous intrusion detection system (AIDS). There are 2 necessary open-source materials involved in the project: a data set and a neuroevolution implementation. The product will use NSL-KDD, a dataset containing network data frequently used in cybersecurity machine learning scenarios. For the neuroevolution implementation HyperNEAT in C++ will be used. HyperNEAT is an extension of the original neuroevolution implementation known as NEAT. Neuroevolution utilizes genetic algorithms and fitness algorithms to develop artificial neural networks (ANNs). The product will use the 2 resources previously listed and fitness algorithms, simple ANNs, and experiment scripts to develop an AIDS.

1.2. Scope

The scope of this project is constrained. The product's learning mechanism will not run in real-time. The dataset and the neuroevolution implementation (NSL-KDD and HyperNEAT) used for testing is a static source of information and made available as an open-source resource. This means that more detailed information is available on the dataset by other institutions and other individuals can attempt to replicate the results using the same open-source material. The software will not train in real-time. The software's input capabilities are limited to csv pre-generated datasets and does not have any packet sniffing or network surveillance capabilities. The software's input capabilities are designed to read a dataset of the csv type and can convert the data to float values for the neural network but will not normalize the data. An unnormalized training dataset may limit the quality of ANN learning. Other considerations have been made when handling the dataset, like managing hope linear relationships are implied when converting the input dataset, that ideally demonstrates a conscious attempt to facilitate productive learning behavior for the ANN. The AIDS capabilities of our project will be constrained to only producing flags in recognition of suspected malware. The software has no malware mitigating capabilities.

1.3. System Overview

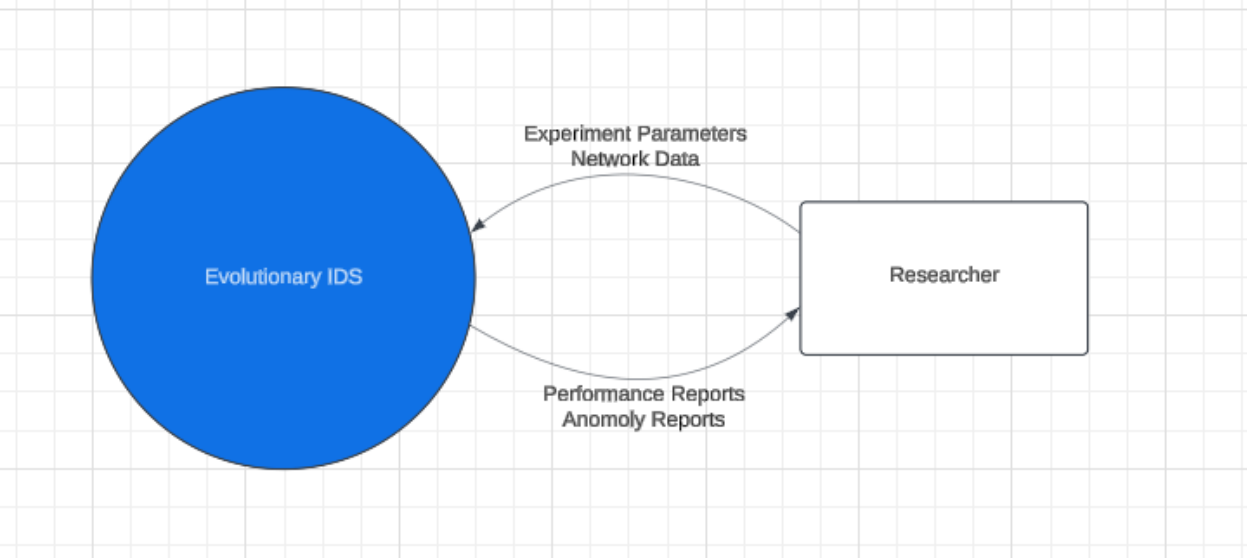


Figure 1. System context diagram

The evolutionary AIDS is a complicated system within itself however it does not interact with external elements outside of the researcher. The system simply intakes network data and experiment parameters then returns training performance reports and anomaly detection reports. In further detail the system accepts data and parameters for training. Prior to execution of the training phase the data will be analyzed and reformatted into a separate CSV so that the evolutionary process has an easier time reading the data. This also attempts to eliminate any linear activity from occurring, for example, the model could assume that there is a closer relationship between UDP and TCP than TCP and ICMP which would cause for improper evolution, potentially causing a loss of anomaly detection accuracy. After the data is correctly formatted the system will use it to begin the evolution process. HyperNEAT will begin evolving compositional pattern producing networks (CPPNs) based on the fitness algorithm for the experiment. The parameters provided by the researcher determine the number of iterations, among other things, which will denote when the process ends. The final product will allow for more data from the CSV to be fed through to determine the success of the evolution.

1.4. Testing Approach Overview

To thoroughly test this system, there are 5 different testing types:

1. **Functional Testing**
    - General tests for verifying the system performs the required functions.
2. **Compatibility Testing**
    - Tests to ensure the system is compatible with software environment and tools.
3. **Dataset Validation Testing**
    - Tests to ensure the formatting and structure of the dataset.
4. **Usability Testing**
    - Tests to ensure the functionality of the user interfaces and user accessibility.

5. **Algorithm Testing**
    - Tests focused on the neural network training and evolutionary algorithms

1.5. Testing Entrance Criteria

The software environment should have all necessary software related to our design, it should have the raw CSV dataset input, a compiler and a building mechanism or an IDE with built-in code running functionalities.

1.6. Document Overview

Section 1: Section 1 contains information regarding the purpose of this document, the scope of the project, a general overview of the system to be tested, an outline for the testing approach, the testing entrance criteria for requirements necessary for testing, and the references for this document.

Section 2: Section 2 is the testing approach. It details much of the planning phase of testing. It includes the testing types, suspension criteria and resumption requirements, the testing environment, assumptions, risks and contingencies for testing and finally a detailed test plan.

Section 3: Section 3 contains the testing schedule.

Section 4: Section 4 contains the traceability matrix and defect tracking. It details which test cases each of the requirements defined in the SRS belong to. The chart for rankings of defects is also included in this section.

Section 5: Section 5 contains the test cases. They examine each individual test in detail, including information regarding test methods, extent, performance metrics, and the actual results of tests.

1.7. References

[1] J. Gauci and K. O. Stanley, “A case study on the critical role of geometric regularity in machine learning,” National Conference on Artificial Intelligence, pp. 628–633, Jul. 2008.

[2] B. Su, R. Li, and H. Zhang, “Evolving Deep Convolutional Neural Network for Intrusion Detection Based on NEAT,” Oct. 2020, doi: https://doi.org/10.1109/wpmc50192.2020.9309451.

[3] K. Ansam et al, "Survey of intrusion detection systems: techniques, datasets and challenges," Cybersecurity, vol. 2, (1), 2019. DOI: https://doi.org/10.1186/s42400-019-0038-7.

[4] K. O. Stanley, J. Clune, J. Lehman, and R. Miikkulainen, “Designing neural networks through neuroevolution,” *Nature Machine Intelligence*, vol. 1, no. 1, pp. 24–35, Jan. 2019, doi: https://doi.org/10.1038/s42256-018-0006-z.

[5] MisterTea, “GitHub - MisterTea/HyperNEAT: The original implementation of HyperNEAT in C++,” *GitHub*, 2024. https://github.com/MisterTea/HyperNEAT (accessed Oct. 18, 2024).

2. Testing Approach

This section outlines the overall strategy and methodology for testing the system. It defines the different types of testing that will be conducted, the specific requirements associated with each type, and the conditions under which testing may be suspended or resumed. Additionally, it details the testing environment, underlying assumptions, and any identified risks or contingencies. The purpose of this structured approach is to ensure comprehensive verification of the system's functionality, compatibility, usability, and performance, while maintaining consistency and traceability throughout the testing lifecycle. Table 1 provides a summary of test executions, including outcomes and dates.

2.1. Testing Types

Listed below are the testing types and associated requirements. There are a few requirements that fall into multiple categories.

2.1.1. Functional Testing

Includes tests that verify the system performs the required functions.

Related Sections from System Requirement Specification document:

- **Dataset Management:** Ensures that datasets can be uploaded, selected, stored, and accessed efficiently throughout the system, supporting smooth user interaction and integration.
- **Dataset Capability:** Verifies that the system can process datasets of varying sizes, structures, and content while ensuring they remain within acceptable limits and are suitable for analysis.
- **Analysis and Reporting:** Confirms that the system can generate accurate performance metrics, summaries, and visual outputs, helping users interpret the outcomes of training and testing.
- **Evolutionary Behavior:** Validates that the evolutionary algorithm functions as expected, including proper handling of initialization, mutation, selection, and generational progress.
- **Error Handling:** Ensures that the system gracefully handles unexpected inputs or failures by generating appropriate error messages and maintaining operational stability.
- **Training and Testing Flow:** Validates the end-to-end process of training the neural network and evaluating its performance, including dataset partitioning and test integrity.

2.1.2. Compatibility Testing

Tests to ensure the software is compatible with software environment and tools

Related Sections from System Requirement Specification document:

- **Operating Systems:** Validates that the software is compatible across the required operating systems, ensuring consistent behavior and performance regardless of the host platform.
- **Libraries and Tools:** Ensures that all external libraries, frameworks, and software tools used by the system are correctly integrated, version-aligned, and functioning as expected within the software environment.
- **Development Environment:** Confirms that the system can be built, executed, and maintained within the designated development environments, including support for IDEs, compilers, and build systems.

2.1.3. Dataset Validation Testing

Tests that have to do with the dataset formatting and structure requirements.

Related Sections from System Requirement Specification document:

- **Dataset Format and Structure:** Verifies that the dataset adheres to the required structural standards, including correct file types, column formats, and consistent data organization.
- **Dataset Content:** Ensures that the data within the dataset is relevant, accurate, and free from invalid or unexpected values that could disrupt processing.
- **Dataset Partitioning and Distribution:** Confirms that datasets are correctly divided into training, validation, and testing subsets, and that the distribution supports fair and effective model evaluation.
- **Dataset Size Limits:** Validates that dataset sizes fall within acceptable thresholds, ensuring the system can handle the data efficiently without performance degradation.
- **Dataset Processibility:** Ensures that the dataset can be fully parsed, loaded, and used by the system for analysis and training without errors or interruptions.

2.1.4. Usability Testing

Tests that have to do with the user interface and user accessibility

Related Sections from System Requirement Specification document:

- **UI Elements:** Ensures that user interface components are present, functional, and intuitive, supporting a smooth and efficient user experience during interaction with the system.
- **Accessibility:** Verifies that the interface is usable by individuals with varying levels of technical experience, ensuring clarity, simplicity, and ease of navigation.
- **Error Messaging:** Confirms that clear and informative error messages are displayed when problems occur, helping users understand and resolve issues effectively.

2.1.5. Algorithm Testing

Tests that verify the functionality of the neural network training and evolutionary algorithms

Related Sections from System Requirement Specification document:

- **Training and Testing Operations:** Validates that the neural network and evolutionary algorithms can be trained and tested correctly, following the expected workflows and producing reliable results.
- **Neural Network Performance:** Ensures that the neural network performs as intended, meeting accuracy and efficiency benchmarks under various testing conditions.
- **Fitness Evaluation:** Confirms that the system accurately evaluates the performance of evolved solutions using appropriate fitness metrics to guide the evolutionary process.

2.2. Testing Suspension Criteria and Resumption Requirements

This section will specify the criteria that will be used to suspend all or a portion of the testing activities on the items associated with this test plan.

2.2.1. Suspension Criteria

Testing will be suspended if the incidents found will not allow further testing of the system/application under-test. If testing is halted, and changes are made to the hardware, software, or database, it is up to the Testing Manager to determine whether the test plan will be re-executed, or part of the plan will be re-executed.

2.2.2. Resumption Requirements

Resumption of testing will occur when suspension criteria have been mitigated. Software can be composed of bugs and system errors that are hard to categorize and predict in detail. If non-terminating loops, error interrupts, library access failures, reference call failures and system freeze or crashes, testing will be suspended. The developers must return to the debug phase Once the Testing Manger comes to a reasonable confidence that debug efforts have been constructive and the system has been restored to its compliable state, testing can resume.

2.3. Testing Environment

It is recommended that testing is carried out inside. The computer hardware running the program should have an adequate supply of power. No Wi-Fi or external interfaces are required. The software environment should have all necessary software related to our design, it should have the raw CSV dataset input, a compiler and a building mechanism or an IDE with build-in code running functionalities

2.4. Testing Assumptions

It is assumed that no user has a malicious intent, and all users seek to use the code correctly, are familiar with the code and will prompt the code in a way that provides the desired and expected outputs. As a result, we are not testing the software for high-level resilience, exception handling or defensive capabilities.

2.5. Testing Risks and Contingencies

While no major risks are anticipated, the following potential issues could impact testing activities. To mitigate these risks, appropriate contingencies and monitoring measures will be in place:

- **Dataset Corruption or Incompatibility**
  Risk: If the dataset becomes corrupted, improperly formatted, or incompatible with the processing pipeline, it may cause test failures or misleading results.
  Contingency: Maintain backup copies of validated dataset and perform integrity checks.
- **Inconsistent or Unstable Test Environment**
  Risk: If the software dependencies, libraries, or development environment change unexpectedly, it may lead to inconsistent test behavior.
  Contingency: Lock and document environment configurations, maintain backups of iterations on GitHub
- **Insufficient System Resources**
  Risk: The hardware used for testing may not meet the resource demands of neural network training or evolutionary processes, leading to crashes or incomplete test execution.
  Contingency: Scale down experiments or optimize model configurations for limited environments; optionally move to more powerful machines for intensive tests.
- **Software Bugs in Core Logic**
  Risk: Undiscovered bugs in dataset handling, neural network logic, or reporting functions may block or misrepresent test outcomes.
  Contingency: Use stepwise testing and isolate components before full integration tests.
- **Unexpected Evolution Behavior**
  Risk: The evolutionary algorithm may converge prematurely, diverge indefinitely, or fail to evolve viable solutions, invalidating training tests.
  Contingency: Monitor fitness progression, modify fitness algorithm, and define fallback strategies to handle unexpected results.

2.6. Test Plan

Table 1: Test Plan (Each test, pass or fail, and date tested)

ID	Test	Status	Date	Notes
1.1	Validate Dataset Management	Pass	03/25/2025	NA
1.2	Validate Analysis and Reporting	Pass	02/20/2025	NA
1.3	Validate Evolutionary Behavior	Pass	02/20/2025	NA
1.4	Validate Error Handling	Pass	04/10/2025	NA
1.5	Validate Training and Testing Flow	Pass	02/20/2025	NA
2.1	Validate Operating System Compatibility	Pass	10/15/2024	NA
2.2	Validate Libraries and Tools Compatibility	Pass	10/15/2024	NA
2.3	Validate Development Environment	Pass	10/15/2024	NA
3.1	Validate Dataset Format and Structure	Pass	04/10/2025	NA
3.2	Validate Dataset Content	Pass	09/21/2024	NA
3.3	Validate Dataset Partitioning and Distribution	Pass	09/21/2024	NA
3.4	Validate Dataset Size Limits	Pending	##/##/####	NA
3.5	Test Dataset Processibility	Pass	03/27/2025	NA
4.1	Validate User Interface Elements	Pass	02/20/2025	NA
4.2	Validate UI Accessibility	Pass	03/26/2025	NA
4.3	Validate Error Messaging in UI	Pass	03/26/2025	NA
5.1	Validate Training and Testing Operations	Pass	03/26/2025	NA
5.2	Validate Neural Network Performance	Pass	03/26/2025	NA
5.3	Validate Fitness Evaluation and Evolution	Pass	03/26/2025	NA

3. Test Schedule

ID	Test	Date
1.1	Validate Dataset Management	03/26/2025
1.2	Validate Analysis and Reporting	02/20/2025
1.3	Validate Evolutionary Behavior	02/20/2025
1.4	Validate Error Handling	04/10/2025
1.5	Validate Training and Testing Flow	02/20/2025
2.1	Validate Operating System Compatibility	10/15/2024
2.2	Validate Libraries and Tools Compatibility	10/15/2024
2.3	Validate Development Environment	10/15/2024
3.1	Validate Dataset Format and Structure	04/10/2025
3.2	Validate Dataset Content	09/21/2024
3.3	Validate Dataset Partitioning and Distribution	09/21/2024
3.4	Validate Dataset Size Limits	##/##/####
3.5	Test Dataset Processibility	03/27/2025
4.1	Validate User Interface Elements	02/20/2025
4.2	Validate UI Accessibility	03/26/2025
4.3	Validate Error Messaging in UI	03/26/2025
5.1	Validate Training and Testing Operations	03/26/2025
5.2	Validate Neural Network Performance	03/26/2025
5.3	Validate Fitness Evaluation and Evolution	03/26/2025

4. Traceability Matrix and Defect Tracking

4.1. Traceability Matrix

Table 2: Traceability Matrix

Req. ID	Test Case(s)
REQ-201	Test Case 2.1
REQ-202	Test Case 2.1
REQ-203	Test Case 2.1
REQ-204	Test Case 2.1
REQ-301	Test Case 1.1, Test Case 4.1
REQ-302	Test Case 1.1, Test Case 4.1

REQ-303	Test Case 4.1
REQ-304	Test Case 1.5, Test Case 4.1
REQ-305	Test Case 1.5, Test Case 4.1
REQ-306	Test Case 1.1, Test Case 4.2
REQ-307	Test Case 4.3
REQ-308	Test Case 2.2
REQ-309	Test Case 2.2
REQ-310	Test Case 2.3
REQ-311	Test Case 2.3
REQ-312	Test Case 2.3
REQ-313	Test Case 2.3
REQ-314	Test Case 2.3
REQ-401	Not directly applicable (general context of the dataset)
REQ-402	Test Case 3.2
REQ-403	Test Case 3.1
REQ-405	Test Case 1.4
REQ-406	Test Case 1.1, Test Case 3.1
REQ-407	Test Case 1.6
REQ-408	Test Case 1.5
REQ-409	Test Case 1.5
REQ-410	Test Case 1.4
REQ-411	Test Case 5.1
REQ-412	Test Case 5.1
REQ-413	Test Case 1.5, Test Case 5.2
REQ-414	Test Case 1.4
REQ-415	Test Case 1.6
REQ-416	Test Case 1.6
REQ-417	Test Case 1.6
REQ-418	Test Case 3.4
REQ-419	Test Case 1.6, Test Case 4.3
REQ-420	Test Case 1.1
REQ-421	Test Case 1.3
REQ-422	Test Case 1.4
REQ-423	Test Case 1.3, Test Case 5.3
REQ-424	Test Case 1.4, Test Case 2.3, Test Case 5.3
REQ-425	Test Case 5.2
REQ-501	Test Case 2.2
REQ-502	Test Case 3.4

REQ-503	Test Case 3.4
REQ-504	Test Case 3.5
REQ-505	Test Case 3.5
REQ-506	Test Case 2.3
REQ-507	Test Case 2.3
REQ-508	Test Case 2.3
REQ-509	Test Case 2.3
REQ-510	Test Case 2.3
REQ-511	Test Case 2.3
REQ-512	Test Case 2.3
REQ-513	Test Case 2.4
REQ-514	Test Case 2.2
REQ-515	Test Case 2.2
REQ-516	Test Case 2.4
REQ-517	Test Case 2.4
REQ-518	Test Case 2.4
REQ-601	Test Case 3.1
REQ-602	Test Case 3.1
REQ-603	Test Case 1.3, Test Case 3.3
REQ-604	Test Case 3.2
REQ-605	Test Case 3.2
REQ-606	Test Case 3.1
REQ-607	Test Case 1.2
REQ-608	Test Case 3.1
REQ-609	Test Case 3.1, Test Case 3.3
REQ-610	Test Case 4.1
REQ-611	Test Case 1.1
REQ-612	Test Case 1.1
REQ-613	Test Case 4.1
REQ-614	Test Case 1.5
REQ-615	Test Case 1.5
REQ-616	Test Case 1.5

4.2. Defect Severity Definitions

Critical	The defect causes a catastrophic or severe error that results in major problems and the functionality is rendered unavailable to the user. A manual procedure is impossible to implement, or high effort is required to remedy the defect. Examples of critical defects are as follows: • System abnormally terminates • Data cannot flow through a business function/lifecycle • Data is corrupted or cannot post to the database
Medium	A defect that does not seriously impair system function can be categorized as a medium defect. A manual procedure requiring medium effort can be implemented to remedy the defect. Examples of medium defects are as follows: • Form navigation is incorrect • Field labels are not consistent with global terminology
Low	The defect is cosmetic or has little to no impact on system functionality. A manual procedure requiring low effort can be implemented to remedy the defect. Examples of low defects are as follows: • Repositioning of fields on screens • Text font on reports is incorrect

5. Test Cases

1. Functional Testing

Test Case 1.1: Verify Dataset Management

Objective: Ensure the system handles dataset insertion, configuration, and management effectively.

Test No.: 1.1			Current Status: Pass	
Test title: Verify Dataset Management				
Testing approach: Functional Testing				
STEPS 1	OPERATOR ACTION Start the software	PURPOSE Initialize settings	EXEPCTED RESULTS Training and testing modes are accessible and functional.	COMMENTS
2	Insert datasets for training and testing.	Create a logical link between the software and the database to instigate processing	The system accepts and manages datasets without errors.	
3	Verify accessibility and usability of inserted datasets.	Enabled the tester to observe an error or passing software progression	No errors are thrown by the software	**Requirements Covered:** REQ-301, REQ-302, REQ-306, REQ-406, REQ-420, REQ-611, REQ-612.
4	Switch between training and testing modes.	To evaluate the file, I/O interface between the software and the database ensures compatibility.	The system differentiates and presents the test and training dataset input	
Concluding Remarks: Desired functionality has been reached.				

Testing Team:	Date Completed:
Krajewski, Pepe, Bossie	03/26/2025

Test Case 1.2: Test Analysis and Reporting Functionality

Objective: Ensure accurate generation and display of analysis, training, and detection reports.

Notes: *Ran example experiment, navigated UI, displayed NN visualization*

Test No.: 1.2			Current Status: Pass	
Test title: Test Analysis and Reporting Functionality				
Testing approach: Functional Testing				
STEPS 1	OPERATOR ACTION Perform training and testing using valid datasets.	PURPOSE Initialization	EXEPCTED RESULTS Reports are accurate and properly displayed.	COMMENTS **Requirements Covered:** REQ-304, REQ-305, REQ-408, REQ-413, REQ-614, REQ-615, REQ-616.
2	Trigger report generation for analysis, training, and testing phases.	Instigate a change in the UI for viewing	The expected user interface is produced	
3	View generated reports in the UI.	Enables the tester to ensure accurate generation and display of analysis, training, and detection reports.	The expected user interface is produced The UI shows expected or reasonable values describing core behaviors	
Concluding Remarks: UI is running and visualizations are accessible				
Testing Team: Donay, Krajewski			Date Completed: 02/20/25	

Test Case 1.3: Validate Evolutionary Behavior

Objective: Ensure neuroevolutionary processes are initiated and executed as expected.

Notes: *Test for functionality required CMake build of HyperNEAT and example experiment.*

Test No.: 1.3			Current Status: Pass	
Test title: Validate Evolutionary Behavior				
Testing approach: Functional Testing				
STEPS 1.	OPERATOR ACTION Start the evolutionary process for training.	PURPOSE Ensure the program produces an evolving NN architecture	EXEPCTED RESULTS Evolutionary processes are executed without error.	COMMENTS **Requirements Covered:** REQ-405, REQ-410, REQ-414, REQ-422, REQ-424.
2	View nodal parameters and fitness metrics upon completion.	Enable the user to verify the UI behavior	Results are displayed correctly.	
Concluding Remarks: The neuroevolutionary process is functional, NNs are evolving and can be viewed.				
Testing Team: Bossie, Donay, Krajewski, Pepe		Date Completed: 02/20/25		

Test Case 1.4: Test Error Handling

Test No.: 1.4			Current Status: Pass	
Test title: Test Error Handling				
Testing approach: Functional Testing				
STEPS 1	OPERATOR ACTION Provide invalid, missing, or unreadable datasets.	PURPOSE Ensure that edge cases for large dataset are sufficiently safeguarded against	EXEPCTED RESULTS Errors are handled as per specifications with appropriate messaging. The system skips invalid entries and continues processing when applicable.	COMMENTS **Requirements Covered:** REQ-407, REQ-415, REQ-416, REQ-417, REQ-418, REQ-419.
2	Attempt to exceed dataset size limits.	Test the edge cases of the dataset size	Errors are handled as per specifications with appropriate messaging. The system skips invalid entries and continues processing when applicable.	
3	Request parameters for nonexistent nodes.	Test the value handling and logic control of node location	The system skips invalid entries and continues processing when applicable.	
4	Use incorrectly formatted dataset entries.	Test the code's ability to differentiate between the defined, appropriate file type	Errors are handled as per specifications with appropriate messaging. The system skips invalid entries and continues processing when applicable.	
Concluding Remarks: The program successfully validates the dataset and ignores invalid entries.				
Testing Team: Krajewski, Pepe			Date Completed: 04/10/2025	

Test Case 1.5: Verify Training and Testing Flow

Test No.: 1.5			Current Status: Pass	
Test title: Verify Training and Testing Flow				
Testing approach: Functional Testing				
STEPS 1	OPERATOR ACTION Use a dataset partitioned into training and testing subsets.	PURPOSE Ensure the training and testing processes follow the specified flow and produce expected results.	EXEPCTED RESULTS Training and testing are completed successfully. Results align with specified criteria.	COMMENTS **Requirements Covered:** REQ-421, REQ-423, REQ-603, REQ-609.
2	Train the neural network for a specified number of generations.	Test the software's ability to pursue an architecture for a set number of iterations	Training and testing are completed successfully. Results align with specified criteria.	
3	Evaluate training results and switch to testing mode.	Test the software's switching functionality, ensure both testing and training have a flow of control	Training and testing are completed successfully. Results align with specified criteria.	
4	Test the neural network using the specified dataset	Instigate a training trial	Training and testing are completed successfully. Results align with specified criteria.	
Concluding Remarks: All the expected results can be viewed through the exported XML, which provides accuracy, fitness and the number of generations.				
Testing Team: Bossie, Donay, Krawjewski, Pepe			Date Completed: 04/03/2025	

Compatibility Testing

Test Case 2.1: Verify Operating System Compatibility

Test No.: 2.1			Current Status: Pass	
Test title: Verify Operating System Compatibility				
Testing approach: Compatibility Testing				
STEPS 1	OPERATOR ACTION Install and run the software on Windows 10 and Windows 11.	PURPOSE Ensure compatibility with supported operating systems.	EXEPCTED RESULTS The software operates seamlessly on both OS versions.	COMMENTS **Requirements Covered:** REQ-308, REQ-514, REQ-515. REQ-418, REQ-419.
2	Execute all functional operations.	Ensure compatibility with supported operating systems.	The software operates seamlessly on both OS versions.	
Concluding Remarks: The program successfully ran on computers with Windows 10 and Windows 11.				
Testing Team: Krajewski, Donay, Pepe, Bossie			Date Completed: 10/15/2024	

Test Case 2.2: Verify Library and Tool Integration

Test No.: 2.2			Current Status: Pass	
Test title: Verify Library and Tool Integration				
Testing approach: Compatibility Testing				
STEPS 1	OPERATOR ACTION Build and run the software with the specified versions of libraries and tools.	PURPOSE Ensure all specified libraries and tools are functional and compatible.	EXEPCTED RESULTS All tools and libraries function correctly without errors.	COMMENTS **Requirements Covered:** REQ-310, REQ-311, REQ-312, REQ-313, REQ-314, REQ-424, REQ-506, REQ-507, REQ-508, REQ-509, REQ-510, REQ-511, REQ-512.
2	Verify operations such as dataset handling, UI functionality, and neural network processes.	Ensure all specified libraries and tools are functional and compatible.	All tools and libraries function correctly without errors.	
Concluding Remarks: Tools function properly				
Testing Team: Krajewski, Donay, Pepe, Bossie			Date Completed: 10/15/2024	

Test Case 2.3: Validate Development Environment Setup

Test No.: 2.3			Current Status: Pass	
Test title: Validate Development Environment Setup				
Testing approach: Compatibility Testing				
STEPS 1	OPERATOR ACTION Set up the development environment with Visual Studio 2022.	PURPOSE Ensure the development environment supports the project's requirements.	EXEPCTED RESULTS Project compiles and runs successfully.	COMMENTS **Requirements Covered:** REQ-516, REQ-517, REQ-518.
2	Compile the project using MSVC 14.3.	Ensure the development environment supports the project's requirements.	Project compiles and runs successfully.	
Concluding Remarks: The environment allows for building, running and compiling the program				
Testing Team: Krajewski, Donay			Date Completed: 10/15/2024	

Dataset Validation Testing

Test Case 3.1: Validate Dataset Format and Structure

Test No.: 3.1			Current Status: Pass	
Test title: Dataset Validation Testing				
Testing approach: Database Validation Testing				
STEPS 1	OPERATOR ACTION Use datasets with valid formats and structures.	PURPOSE Ensure datasets meet format and structural requirements.	EXEPCTED RESULTS Proper datasets are accepted, and improperly formatted datasets are rejected with appropriate error messages.	COMMENTS **Requirements Covered:** REQ-403, REQ-406, REQ-608, REQ-609, REQ-606.
2	Provide datasets with missing columns or labels.	To produce an error case related to dataset structure	Proper datasets are accepted, and improperly formatted datasets are rejected with appropriate error messages.	
3	Request parameters for nonexistent nodes.	Test the value handling and logic control of node location	Proper parameter files are accepted, and illogical parameters are rejected with appropriate error messages.	
4	Use incorrectly formatted dataset entries.	Test the edge cases of the dataset	Proper datasets are accepted, and improperly formatted datasets are rejected with appropriate error messages.	
Concluding Remarks: Program successfully validates datasets and ignores invalid entries.				
Testing Team: Krajewski, Pepe			Date Completed: 04/10/2025	

Test Case 3.2: Validate Dataset Content

Test No.: 3.2			Current Status: Pass	
Test title: Validate Dataset Content				
Testing approach: Database Validation Testing				
STEPS 1.	OPERATOR ACTION Analyze a provided dataset to ensure it includes events like DoS and routine network actions.	PURPOSE Ensure datasets contain necessary malicious and normal events.	EXEPCTED RESULTS The dataset contains both malicious and normal actions as required.	COMMENTS **Requirements Covered:** REQ-402, REQ-604, REQ-605.
Concluding Remarks: Training and testing partitions contain DoS, Probe, R2L and U2R malware instances. Verified by Reviewing the Labels in Excel.				
Testing Team: Bossie			Date Completed: 9/21/2024	

Test Case 3.3: Validate Dataset Partitioning and Distribution

Test No.: 3.3			Current Status: Pass	
Test title: Validate Dataset Partitioning and Distribution				
Testing approach: Database Validation Testing				
STEPS 1	OPERATOR ACTION Partition a dataset into training and testing subsets.	PURPOSE Ensure datasets are correctly partitioned and distributed.	EXEPCTED RESULTS Dataset partitions meet the specified criteria.	COMMENTS **Requirements Covered:** REQ-603, REQ-609.
2	Verify that both subsets maintain a representative distribution of malicious and normal actions.	Ensure datasets are correctly partitioned and distributed.	Dataset partitions meet the specified criteria.	
Concluding Remarks: Training set was analyzed for the distribution of normal and malicious instances. A table and pie chart were created from the first semester poster presentation. This was a verification of our pre-partitioned open-source dataset.				
Testing Team: Bossie, Pepe			Date Completed: 9/21/2024	

Test Case 3.4: Validate Dataset Size Limits

Test No.: 3.4			Current Status: Pending	
Test title: Validate Dataset Size Limits				
Testing approach: Database Validation Testing				
STEPS 1.	OPERATOR ACTION Use a dataset with more than 1,000,000 entries.	PURPOSE Test the system's behavior with datasets of varying sizes.	EXEPCTED RESULTS The system processes up to 999,999 entries and reports an error for excess entries.	COMMENTS **Requirements Covered:** REQ-418, REQ-502, REQ-503.
Concluding Remarks: << Filled in by the person who completed the test >>				
Testing Team: Pepe, Bossie			Date Completed:	

Test Case 3.5: Test Dataset Processibility

Test No.: 3.5			Current Status: Pass	
Test title: Test Dataset Processibility				
Testing approach: Database Validation Testing				
STEPS 1.	OPERATOR ACTION Train and test the neural network with a valid dataset.	PURPOSE Instigate the desired runtime behavior for measuring	EXEPCTED RESULTS The program runs as outlines in the requirements	COMMENTS **Requirements Covered:** REQ-504, REQ-505.
2.	Measure processing speeds for training and testing.	Verify dataset processing meets performance standards.	Training processes at least 100 entries per second. Testing processes at least 25 entries per second.	
Concluding Remarks: Based on the calculated timeframe between generations, the program is processing dataset entries at the desired rates.				
Testing Team: Bossie, Donay, Krajewski, Pepe		Date Completed: 03/27/2025		

Usability Testing

Test Case 4.1: Validate UI Element Functionality

Test No.: 4.1			Current Status: Pass	
Test title: Validate UI Element Functionality				
Testing approach: Usability Testing				
STEPS 1.	OPERATOR ACTION Start the software.	PURPOSE Initialize	EXEPCTED RESULTS Software starts and runs according to requirement defined behaviors	COMMENTS
2	Interact with dataset insertion, analysis, and reporting features.	Ensure all UI elements are functional and accessible.	All UI elements respond correctly.	**Requirements Covered:** REQ-301, REQ-302, REQ-303, REQ-304, REQ-305.
Concluding Remarks: Software can start and initialize as expected.				
Testing Team: Bossie, Donay, Krajewski, Pepe			Date Completed: 02/20/2025	

Test Case 4.2: Test Accessibility and Response Times

Test No.: 4.2			Current Status: Pass	
Test title: Test Accessibility and Response Times				
Testing approach: Usability Testing				
STEPS 1.	OPERATOR ACTION Measure the time taken to access UI elements after startup.	PURPOSE Ensure the UI is responsive.	EXEPCTED RESULTS UI elements load within 30 seconds.	COMMENTS **Requirements Covered:** REQ-306, REQ-307, REQ-419.
2	Trigger an error and measure response time.	Ensure error messages are responsive.	Errors are displayed within 5 seconds.	
Concluding Remarks: All UI elements load within 30 seconds. Error messages appear within 5 seconds typically, but not all errors/unintended behaviors produce error messages.				
Testing Team: Donay, Krajewski			Date Completed: 03/26/2025	

Algorithm Testing

Test Case 5.1: Validate Training and Testing Operations

Test No.: 5.1			Current Status: Pass	
Test title: Validate Training and Testing Operations				
Testing approach: Algorithm Testing				
STEPS 1.	OPERATOR ACTION Train a neural network using a valid dataset.	PURPOSE Ensure training operation functions correctly.	EXEPCTED RESULTS Training is completed without errors.	COMMENTS **Requirements Covered:** REQ-411, REQ-412, REQ-423, REQ-424.
2	Test the trained neural network.	Ensure testing operation functions correctly.	Testing is completed without errors.	
Concluding Remarks: Testing and training modes compiled and ran without any errors.				
Testing Team: Krajewski, Pepe			Date Completed: 03/26/2025	

Test Case 5.2: Verify Neural Network Performance

Test No.: 5.2			Current Status: Pass	
Test title: Verify Neural Network Performance				
Testing approach: Algorithm Testing				
STEPS 1.	OPERATOR ACTION Run a testing phase on trained networks.	PURPOSE initialize	EXEPCTED RESULTS Program runs normally	COMMENTS
2.	Evaluate detection reports.	Ensure the neural network correctly flags anomalous activities.	Reports correctly identify anomalous and normal activities.	**Requirements Covered:** REQ-413, REQ-425.
Concluding Remarks: Algorithm can successfully evolve to improve intrusion detection rates (with fitness as a metric).				
Testing Team: Krajewski, Pepe			Date Completed: 03/26/2025	

Test Case 5.3: Validate Fitness Evaluation

Test No.: 5.3			Current Status: Pass	
Test title: Validate Fitness Evaluation				
Testing approach: Algorithm Testing				
STEPS 1.	OPERATOR ACTION Define a fitness function.	PURPOSE Alter the code so that the software exhibits the desired fitness selective behavior	EXEPCTED RESULTS Code compiles	COMMENTS
2.	Train the neural network using this function.	Ensure user-defined fitness functions evaluate neural network performance.	Neural network performance aligns with the fitness criteria.	**Requirements Covered:** REQ-423.
Concluding Remarks: Program successfully compiles, runs and produces an evolving network that improves accuracy.				
Testing Team: Krajewski, Pepe			Date Completed: 03/26/2025	